Group # 13 A: Yuqi Lao, Renzon Gabriel

Additional Documentation

Chapter 1: Design and Implementation

1. Move the function of the snake (discuss speed and how it moves)

The snake is initially stored as a list of (x,y) tuples moving left. The rightmost tuple is the first element of the list. Since the initial snake x-coordinates were all 10 pixels apart, we decided for the calculateNewCoordinates function to move 10 pixels for every step the snake takes. For the move function, we add these new snake coordinates to the list, while constantly maintaining the length of the snake by removing its tail with the pop(0) command. On the other hand, when it eats the prey, it doesn't call the pop(0) command, which makes the snake 10 pixels longer.

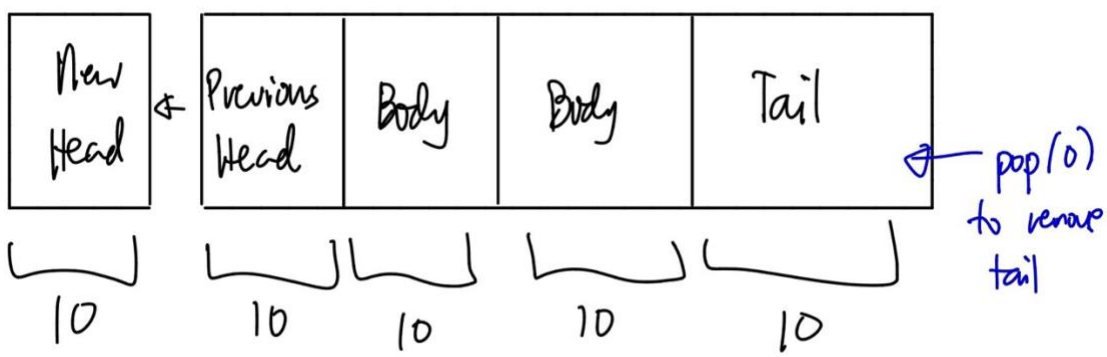

Figure 1.1 : Visual representation of the snakeCoordinates list moving left

2. Prey Capture, Prey Coordinates, Prey Thresholds and Generating New Prey

For prey capture, we decided to approach the game similarly to Chess. In Chess, as every center piece has its own square, if another piece enters and eats it, the other piece only needs to enter the square as well. Same ideology as the snake and prey, since preyPosition is the center of the entire prey, the head of the snake would be the other chess piece that enters the square and gains a point. To solve this problem, we used the Chebyshev Distance formula, which calculates the distance between two points. For this, we use the preyPosition center coordinates and the head coordinates of the snake. The Chebyshev Distance formula calculates the absolute value of these two points, and checks if it is under distance d, shown with the formula below in Figure 2.1. Since d is the distance between centers, we sum the widths of the two objects and divide by 2. Similar logic applies to generating new prey, ensuring the prey doesn't touch the snake. The constants used are DISTANCE_FROM_SNAKE and EAT_PREY_DISTANCE.

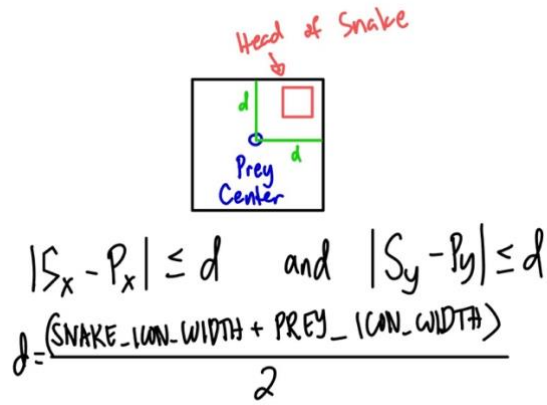

Figure 2.1 : Visual representation of calculating DISTANCE_FROM_SNAKE and EAT_PREY_DISTANCE constants

For prey coordinates, the diagram shown on Figure 2.2 shows a visual representation of how we chose the constant to be PREY_ICON_WIDTH/2, since the preyPosition of x and y is the center of the prey itself, and following tkinter rectangular format, we decided the order of the preyCoordinates to be from (x0, y0) to (x1,y1).

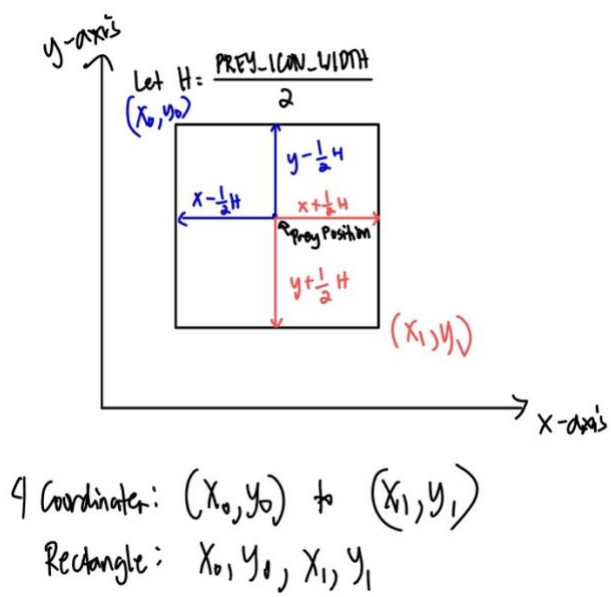

Figure 2.2 : Visual representation of calculating preyCoordinates

For prey thresholds, in choosing random x and y coordinates, we want to ensure the prey doesn't cross the walls. Therefore, with Figure 2.3, this shows a visual representation of how choosing a random integer between 0 + THRESHOLD to WINDOW_SIZE – THRESHOLD.

Figure 2.2 : Visual representation of choosing range of random x and y prey coordinates

For generating new prey, we chose to ensure not appear behind the score text, and not where the snake is currently moving. To ensure it doesn't appear behind the score, we estimated the coordinates where the score text is located with the help of the given scoreTextXLocation and scoreTextYLocation constants. Since the text doesn't extend to the middle of the x-axis, we estimated 200. Meanwhile, for the y-axis, since the font-size is 11, we added scoreTextYLocation + 11 = 26.

3. Chosen GUI constants

We chose SNAKE_ICON_WIDTH to be 10 pixels, as the snakeCoordinates list are also all 10 pixels apart, we chose PREY_ICON_WIDTH to be 7 pixels, we wanted the prey to be smaller than the snake, but more than half of the width of the snake, so we chose 7 pixels. We kept the window height and width to be consistent.

Chapter 2: Challenges

1. Prey Capture

It was pretty challenging and ambiguous at first to decide how we would approach the problem of when to consider eating the prey, but we decided that as long as it touches the prey, a point is given. Initially, we had PREY_ICON_WIDTH and SNAKE_ICON_WIDTH to be the same, and setting the same number for EAT_PREY_DISTANCE was pretty helpful in calculating our distance constant.

2. Prey Appearance on Unwanted Spots

We wanted to ensure it doesn't appear where the snake and score are located. Since x and y coordinates are randomized for testing, it is pretty uncertain how long we would confirm our code logic is correct. Therefore, we temporarily decreased the window_height maximally and modify speed to be quicker for faster testing and debugging.

3. Improvements (various speeds, better threading mechanism)

In the future, since the snake does not move smoothly 100% of the time, maybe various speeds can help with the smoothness of the program, and that could be implemented in the future. Also, adding better threading mechanisms could help with that problem.